

menler

Your turning point in the AI era.

AI APTITUDE QUESTION BANK

LLM

Fundamentals

About This Question Bank

These questions cover the foundational concepts behind large language models: transformer architecture, tokenisation and embeddings, context windows, inference mechanics, and model selection.

5

Sets

15

Questions per Set

75

Total Questions

SET GUIDE

Set 1 · How Language Models Work

Transformer architecture — attention, layers, residuals, and model scale

Set 2 · Tokens and Embeddings

Tokens and embeddings — how text becomes numbers and what those numbers mean

Set 3 · Context Windows

Context windows — limits, management, and practical implications

Set 4 · Inference and Reasoning

Inference and reasoning — how models generate, sample, and think

Set 5 · Model Selection and Fine-tuning

Model selection and fine-tuning — choosing and adapting models for production

Each question has one correct answer. Select the best answer from the options provided.

Correct answers and explanations follow each question.

SET

1

How Language Models Work

Set 1 of 5 · 15 Questions · LLM Fundamentals

Q1. What is the core mechanism that allows transformer models to weigh the relevance of different parts of an input when producing an output?

- A. Recurrent processing, which reads tokens one at a time and updates a hidden state.
- B. Self-attention, which computes relationships between all token positions simultaneously.**
- C. Convolutional filters, which detect local patterns across fixed-size windows.
- D. Hard-coded routing tables that assign each token to a specialist sub-network.

Correct Answer: B

Explanation

The transformer architecture is built around self-attention. Every token can attend to every other token in a single pass, allowing the model to weigh relevance across the full input regardless of distance. This is what replaced recurrent processing and enabled parallelism.

Q2. Why did transformers largely replace RNNs for language modelling tasks?

- A. Transformers use less memory per token because they compress sequences into a single vector.
- B. Transformers are deterministic, so they always produce the same output for the same input.
- C. Transformers were designed specifically for text, whereas RNNs were designed for time-series data.
- D. Transformers process all tokens in parallel, making training faster and avoiding the vanishing gradient problem.**

Correct Answer: D

Explanation

RNNs process tokens sequentially, which is slow to train and suffers from vanishing gradients over long sequences. Transformers process the full sequence in parallel via attention, which is both faster to train and better at capturing long-range dependencies.

Q3. In a transformer, what does the feed-forward layer do after the attention layer?

- A. It applies a non-linear transformation independently to each token position.**
- B. It re-orders the token sequence based on predicted importance scores.
- C. It compresses the full sequence into a single context vector.
- D. It retrieves relevant facts from an external knowledge base.

Correct Answer: A

Explanation

After attention mixes information across positions, the feed-forward layer processes each position independently using a two-layer MLP with a non-linear activation. This is where much of the model's "knowledge" is believed to be stored.

Q4. What does the term "parameters" refer to in a large language model?

- A. The configuration settings a developer passes to the model at runtime.
- B. The number of tokens the model can process in a single call.
- C. The learned numerical weights that determine how the model transforms inputs into outputs.**
- D. The hyperparameters used during training such as learning rate and batch size.

Correct Answer: C

Explanation

Parameters are the internal weights of the model — billions of floating-point numbers learned during training. They are distinct from runtime configuration (like temperature or max tokens), context length, or training hyperparameters.

Q5. What is the role of positional encoding in a transformer?

- A. It encodes the semantic meaning of each token into a fixed numerical vector.
- B. It injects information about the order of tokens, since attention itself is position-agnostic.**
- C. It determines which tokens are masked during the training process.
- D. It compresses the full input sequence into a lower-dimensional representation.

Correct Answer: B

Explanation

Attention treats inputs as a set with no inherent order. Positional encoding adds a signal — either fixed sinusoidal or learned — that tells the model where each token sits in the sequence. Without it, the model cannot distinguish "cat sat on mat" from "mat on sat cat".

Q6. What distinguishes a decoder-only transformer (like GPT-style models) from an encoder-decoder transformer (like T5)?

- A. Decoder-only models generate text autoregressively; encoder-decoder models encode an input first, then decode to a target sequence.**
- B. Decoder-only models can only answer questions; encoder-decoder models can generate free-form text.
- C. Decoder-only models are smaller and faster; encoder-decoder models are larger and more accurate.
- D. Decoder-only models require fine-tuning for every task; encoder-decoder models work zero-shot.

Correct Answer: A

Explanation

In decoder-only models, all processing is autoregressive — the model generates one token at a time, attending only to previous tokens. Encoder-decoder models first encode the full input into a representation, then a separate decoder generates the output — useful for translation and summarisation.

Q7. What is "causal masking" in a decoder-only transformer?

- A. Hiding sensitive tokens in the input to prevent the model from memorising private data.
- B. Masking low-confidence tokens in the output to reduce hallucination.
- C. Blocking certain attention heads from activating to reduce compute cost.
- D. Preventing each token from attending to future tokens during training, so the model learns to predict the next token.**

Correct Answer: D

Explanation

Causal masking enforces the autoregressive constraint: during training, each token can only attend to itself and tokens before it. This ensures the model learns to predict the next token given only what came before — not by "seeing ahead".

Q8. What is a "head" in multi-head attention?

- A. The first token in a sequence, which carries the most context for the rest.
- B. A separate sub-network that handles a specific language task like NER or sentiment.
- C. An independent attention function that learns to focus on different aspects of the input simultaneously.**
- D. A parameter group that is frozen during fine-tuning to preserve pre-trained knowledge.

Correct Answer: C

Explanation

Multi-head attention runs several attention functions (heads) in parallel, each with its own learned weight matrices. Different heads can specialise — one might track syntax, another coreference, another topic. Their outputs are concatenated and projected.

Q9. Why do larger language models tend to perform better on tasks they were not explicitly trained for?

- A. Scale enables emergent capabilities — patterns learned across diverse training data generalise to novel tasks.**
- B. Larger models have more memory and can store more facts verbatim from training data.
- C. Larger models are always trained on more recent data, giving them better world knowledge.
- D. Larger models apply explicit reasoning rules, whereas smaller models only match patterns.

Correct Answer: A

Explanation

Emergent capability is a well-documented phenomenon: as models scale in parameters and data, they develop abilities that smaller models do not show — often without those abilities being explicitly trained. This is believed to result from richer internal representations formed by learning across diverse data.

Q10. What is the difference between pre-training and instruction tuning?

- A. Pre-training is done by the user; instruction tuning is done by the model provider.
- B. Pre-training uses labelled data; instruction tuning uses unlabelled internet text.
- C. Pre-training produces a chat model; instruction tuning produces a base model.
- D. Pre-training learns general language representations from raw text; instruction tuning aligns the model to follow human instructions.**

Correct Answer: D

Explanation

Pre-training is the large-scale self-supervised stage where the model learns from raw text by predicting tokens. Instruction tuning (often with RLHF or similar) comes after — it teaches the model to follow instructions, be helpful, and behave appropriately in conversation.

Q11. What does "temperature" control in an LLM at inference time?

- A. The speed of token generation — higher temperature makes the model generate faster.
- B. The sharpness of the probability distribution over next tokens — higher temperature produces more varied outputs.**
- C. The maximum length of the output — higher temperature allows longer responses.
- D. The number of attention heads active during generation — higher temperature activates more heads.

Correct Answer: B

Explanation

Temperature scales the logits before the softmax is applied. At temperature 1.0, the distribution is unchanged. Lower temperature sharpens it (more deterministic); higher temperature flattens it (more random and creative). It does not affect speed, length, or the number of active heads.

Q12. What is a "mixture of experts" (MoE) architecture?

- A. A training technique where multiple smaller models vote on the correct output before it is returned.
- B. A deployment pattern where different model versions handle different user request types.
- C. A model architecture where only a subset of specialist sub-networks are activated for each token, reducing compute per forward pass.**
- D. A fine-tuning approach that trains a separate expert for each target domain.

Correct Answer: C

Explanation

In MoE, the model has many expert sub-networks but only routes each token to a small number of them via a gating mechanism. This allows a much larger total parameter count without proportionally higher compute cost per token — used in models like Mixtral and reportedly GPT-4.

Q13. What is the significance of the "residual connection" in transformer blocks?

- A. It saves the model's previous response so it can refer to it in multi-turn conversations.
- B. It connects the encoder and decoder in sequence-to-sequence models.
- C. It stores intermediate attention scores for use in interpretability tooling.
- D. It adds the block's input directly to its output, helping gradients flow and enabling deeper networks.**

Correct Answer: D

Explanation

Residual (skip) connections add the input of a sub-layer directly to its output. This means gradients can flow through the network without passing through every transformation, enabling very deep models to train stably — a technique borrowed from ResNets.

Q14. What does "RLHF" stand for and what problem does it solve?

- A. Recursive Language Hierarchy Formation — structuring the model's internal representations for better retrieval.
- B. Reinforcement Learning from Human Feedback — aligning model outputs with human preferences beyond what next-token prediction alone achieves.**
- C. Reduced Latency High Fidelity — a technique for speeding up inference without quality loss.
- D. Regularised Loss with Hallucination Filtering — suppressing fabricated outputs during training.

Correct Answer: B

Explanation

RLHF is Reinforcement Learning from Human Feedback. After pre-training, a reward model is trained on human preference data, then used to fine-tune the LLM via RL to produce outputs humans rate more highly. It is the primary technique behind the helpfulness and safety of modern chat models.

Q15. What is "weight sharing" in the context of transformer models?

- A. Using the same parameter matrix for multiple roles in the architecture — for example, tying input embeddings to the output projection layer.**
- B. Distributing a single model's weights across multiple GPUs for parallel inference.
- C. Sharing fine-tuned weights between different user-facing deployments of the same base model.
- D. Copying weights from a larger model into a smaller model as a form of distillation.

Correct Answer: A

Explanation

Weight sharing refers to reusing the same parameter matrix for different purposes. A common example is tying the token embedding matrix to the output linear layer — reducing total parameter count while maintaining performance. It is distinct from model parallelism, deployment sharing, or distillation.

SET

2

Tokens and Embeddings

Set 2 of 5 · 15 Questions · LLM Fundamentals

Q16. What is a token in the context of a large language model?

- A. A security credential required to authenticate API requests.
- B. A discrete reasoning step the model takes before producing an answer.
- C. The basic unit of text the model processes — roughly a word, sub-word, or punctuation mark.**
- D. A numerical identifier assigned to each unique sentence in the training corpus.

Correct Answer: C**Explanation**

A token is the atomic unit of text for an LLM. Models do not process characters or words directly — they process tokens, which are typically sub-word pieces. The word "unbelievable" might be three tokens; "cat" is usually one. Everything — input and output — is counted in tokens.

Q17. Why do some words cost more tokens than others?

- A. Rare or long words are split into multiple sub-word tokens by the tokeniser.**
- B. Common words are cached and cost zero tokens to process.
- C. Words in capitals are always treated as single tokens regardless of length.
- D. Token cost depends on word frequency in the training data, not word length.

Correct Answer: A**Explanation**

Tokenisers like BPE (Byte-Pair Encoding) build a vocabulary of common sub-word pieces. Frequent short words get their own token; rare or long words are split into multiple tokens. "Microprocessor" costs more tokens than "cat" — and this directly affects context usage and cost.

Q18. What is an embedding in the context of LLMs?

- A. A compressed version of the model's weights used for faster inference.
- B. A structured format for injecting external data into the model's context.
- C. A technique for reducing hallucinations by anchoring outputs to source text.
- D. A dense numerical vector that represents the meaning of a token or text in a high-dimensional space.**

Correct Answer: D

Explanation

An embedding is a dense vector of floating-point numbers. Each token maps to an embedding vector; the model learns these representations during training such that semantically similar tokens cluster together in the vector space. Embeddings are the bridge between raw text and numerical computation.

Q19. What does it mean for two texts to have a "high cosine similarity" in embedding space?

- A. They share a high proportion of identical tokens after tokenisation.
- B. Their embedding vectors point in nearly the same direction, indicating semantic similarity.**
- C. They were retrieved from the same document in the training corpus.
- D. Their token counts are within a small percentage of each other.

Correct Answer: B

Explanation

Cosine similarity measures the angle between two vectors. A value close to 1 means the vectors point in the same direction — the texts are semantically similar. This is the basis of semantic search and RAG retrieval: finding chunks whose meaning is close to the query, regardless of exact word match.

Q20. What is the difference between a "token embedding" and a "sentence embedding"?

- A. Token embeddings are used during training; sentence embeddings are used only at inference time.
- B. Token embeddings are high-dimensional; sentence embeddings are always 128 dimensions.
- C. Token embeddings represent individual tokens; sentence embeddings represent the meaning of an entire passage as a single vector.**
- D. Token embeddings are learned; sentence embeddings are computed using hand-crafted rules.

Correct Answer: C

Explanation

Token embeddings are per-token vectors — they are the input representations that evolve through transformer layers. Sentence (or passage) embeddings collapse a full text into one vector, often by pooling over token representations. Sentence embeddings are what you store in a vector database for retrieval.

Q21. Why does tokenisation of non-English text often cost more tokens per word?

- A. Non-English models use a different API endpoint that charges at a higher rate.
- B. Non-Latin scripts require Unicode escaping, which multiplies the byte count.
- C. Non-English words are always treated as unknown tokens and replaced by a placeholder.
- D. Tokenisers trained on English-heavy data have larger vocabulary coverage for English, so non-English words are split into more sub-word pieces.**

Correct Answer: D

Explanation

BPE tokenisers build their vocabulary based on the training corpus. If the corpus is English-heavy, frequent English words get their own tokens. Non-English words — especially in morphologically rich languages or non-Latin scripts — are broken into many small pieces, increasing token count for the same semantic content.

Q22. What is the "vocabulary size" of a language model?

- A. The total number of distinct tokens the model recognises — typically tens of thousands of sub-word pieces.**
- B. The number of unique concepts the model can discuss, inferred from its training data.
- C. The maximum number of tokens the model can generate in a single response.
- D. The number of languages the model was trained on.

Correct Answer: A

Explanation

Vocabulary size is the number of entries in the tokeniser's lookup table — each a distinct token (sub-word, word, or special symbol). GPT-4 uses ~100K tokens; other models use 32K–256K. It determines the output layer size and affects how efficiently different languages and domains are represented.

Q23. What happens to embedding representations as they pass through transformer layers?

- A. They remain identical at each layer; layers only affect the attention weights.
- B. They are progressively refined — each layer transforms the representations to capture increasingly abstract and contextual information.**
- C. They collapse to a single value per token that represents its classification label.
- D. They are discarded after each layer and re-computed from the original token IDs.

Correct Answer: B

Explanation

Each transformer layer updates the token representations through attention (mixing information across positions) and the feed-forward layer (per-position transformation). Early layers capture surface features; later layers capture abstract meaning, syntax, and reasoning — building richer representations with depth.

Q24. What is the purpose of "special tokens" like [CLS], [SEP], or <|endoftext|>?

- A. They are placeholders for sensitive information that has been redacted from the input.
- B. They indicate tokens that the model should copy verbatim without modification.
- C. They are error codes inserted when the tokeniser cannot parse a word.
- D. They serve as structural markers that help the model understand task boundaries, sequence starts/ends, or signal generation completion.**

Correct Answer: D

Explanation

Special tokens are not natural language — they are structural signals in the token sequence. [CLS] was used in BERT as a classification token; [SEP] separated segments; <|endoftext|> tells a generative model the sequence has ended. Instruction-tuned models use special tokens to mark system prompts, user turns, and assistant turns.

Q25. What is "semantic search" and how does it differ from keyword search?

- A. Semantic search is faster because it uses pre-computed index tables; keyword search is slower because it reads raw documents.
- B. Semantic search only works on structured data; keyword search works on both structured and unstructured text.
- C. Semantic search matches based on meaning using embedding similarity; keyword search matches based on exact or fuzzy token overlap.**
- D. Semantic search requires a fine-tuned model; keyword search works with any off-the-shelf model.

Correct Answer: C

Explanation

Keyword search looks for matching terms — it finds "car" when you search "car". Semantic search converts query and documents into embedding vectors and retrieves by cosine similarity — it finds "automobile" and "vehicle" when you search "car" because they cluster nearby in embedding space.

Q26. What does it mean when a model is described as having a "128K token context window"?

- A. The model was trained on 128,000 documents.
- B. The model can process up to 128,000 tokens — input plus output — in a single call.**
- C. The model can remember up to 128,000 previous conversations.
- D. The model generates outputs of exactly 128,000 tokens before stopping.

Correct Answer: B

Explanation

128K context window means the model can hold up to 128,000 tokens in its active working memory for a single API call — combining system prompt, conversation history, retrieved documents, and the response. It says nothing about training data volume or memory across calls.

Q27. Why do embeddings from one model not work directly with embeddings from a different model?

- A. Each model learns its own vector space with different dimensions and geometric structure; cross-model distances are meaningless.**
- B. Embeddings are encrypted at generation time and can only be decrypted by the originating model.
- C. Different models use incompatible file formats that cannot be loaded by each other's inference engines.
- D. Embeddings are only valid during the session in which they were generated.

Correct Answer: A

Explanation

Embedding vectors are coordinates in a space that a specific model learned during training. The geometry — which directions correspond to which meanings — is unique to each model. Mixing vectors from different models is like mixing GPS coordinates from two different map projections: the numbers are incompatible.

Q28. What is the role of the embedding layer at the start of a transformer?

- A. It filters out low-relevance tokens before they enter the attention mechanism.
- B. It compresses the input sequence to reduce compute cost in later layers.
- C. It converts token IDs into dense continuous vectors that the attention mechanism can operate on.**
- D. It assigns each token a confidence score that propagates through the network.

Correct Answer: C

Explanation

The embedding layer is a lookup table: given a token ID (an integer), it returns the corresponding embedding vector (a dense float array). This is the first transformation — from discrete symbols to continuous representations that the mathematical operations in attention and feed-forward layers can process.

Q29. What does "dimensionality" refer to in the context of embedding vectors?

- A. The number of languages the embedding model was trained on.
- B. The maximum text length that can be embedded in a single call.
- C. The precision (float16 vs float32) used to store the embedding values.
- D. The number of values in the vector — higher dimensionality allows richer representations but increases storage and compute cost.**

Correct Answer: D

Explanation

Dimensionality is simply the length of the vector — how many numbers it contains. Text embedding models commonly use 768, 1536, or 3072 dimensions. Higher dimensionality can encode more nuance but costs more to store, compute, and index in a vector database.

Q30. What is "out-of-vocabulary" (OOV) handling in modern tokenisers?

- A. OOV tokens are replaced with a fixed [UNKNOWN] token that the model treats as a generic placeholder.
- B. Sub-word tokenisers like BPE eliminate OOV by breaking any word into known sub-word pieces, including byte-level fallbacks.**
- C. OOV handling requires the user to manually add new tokens to the vocabulary before inference.
- D. Modern models skip OOV tokens silently during generation to avoid errors.

Correct Answer: B

Explanation

Classic word-level vocabularies had OOV problems. Sub-word tokenisers like BPE, WordPiece, and SentencePiece solve this by decomposing any word into smaller known pieces. Byte-level BPE (used by GPT models) goes further — any byte sequence can be represented, so OOV is impossible.

SET

3

Context Windows

Set 3 of 5 · 15 Questions · LLM Fundamentals

Q31. What practical problem does a limited context window create for enterprise AI applications?

- A. Long documents, conversation histories, or knowledge bases may not fit, forcing truncation or chunking strategies.**
- B. The model refuses to answer questions about topics not covered in the first 1,000 tokens.
- C. Enterprise users pay per context window slot, making long inputs prohibitively expensive.
- D. Context limits only affect the output length, not the amount of input the model can read.

Correct Answer: A**Explanation**

Context window limits mean you cannot simply feed the model an entire document library or long conversation history. Engineers must decide what to include — through chunking, retrieval, summarisation, or sliding windows — because anything outside the window is invisible to the model.

Q32. What is the "lost in the middle" problem with long context windows?

- A. When context windows are long, the model loses track of the user's original question.
- B. Content placed in the middle of the context is automatically summarised and compressed.
- C. Models tend to pay more attention to content at the beginning and end of the context, and less to content in the middle.**
- D. Long contexts cause the model to switch languages or styles midway through the response.

Correct Answer: C**Explanation**

Research has shown that LLMs are better at using information placed at the start or end of the context than information buried in the middle. This has practical implications for RAG: where you place retrieved chunks matters, not just whether they fit.

Q33. Why does increasing context window size increase inference cost?

- A. Longer contexts require more fine-tuning passes to update the model's weights.
- B. Attention computation scales quadratically with sequence length, making longer contexts significantly more expensive.**
- C. APIs charge a flat fee per context window size tier regardless of actual token count.
- D. Longer contexts increase the model's temperature, requiring more sampling passes.

Correct Answer: B

Explanation

Self-attention computes relationships between every pair of token positions. For a sequence of length n , that is n^2 operations. Doubling the context length quadruples the attention compute. This is why long-context inference is expensive and why efficient attention variants (FlashAttention, sparse attention) matter.

Q34. What is a "sliding window" approach to handling documents longer than the context window?

- A. Expanding the model's context window dynamically as more tokens are added.
- B. Summarising earlier parts of the document and injecting summaries into later windows.
- C. Routing different sections of the document to different model instances simultaneously.
- D. Processing the document in overlapping chunks that each fit within the context, then aggregating results.**

Correct Answer: D

Explanation

When a document exceeds the context window, one approach is to slide a fixed-size window across it — processing each chunk with some overlap to avoid cutting off information at boundaries. Results (answers, summaries, extractions) are then merged. Simple but can miss cross-chunk reasoning.

Q35. What is the difference between "context length" and "memory" for a language model?

- A. Context length is the tokens available in a single call; memory refers to mechanisms for persisting information across multiple calls.**
- B. Context length is a hardware limit; memory is a software configuration set by the developer.
- C. Context length applies to the input; memory applies only to the model's output generation.
- D. Context length and memory are the same thing — different terms used by different providers.

Correct Answer: A

Explanation

Context length is a per-call limit — what the model can see in one inference request. Language models have no built-in memory across calls; once a session ends, the model starts blank. Memory across calls is an application-layer concern achieved through databases, summarisation, or explicit memory tools.

Q36. What happens when you exceed a model's context window?

- A. The model automatically summarises earlier content to make room for new tokens.
- B. The API typically truncates the input, raises an error, or the model silently ignores overflow tokens depending on implementation.**
- C. The model switches to a retrieval mode, fetching the overflow content from a cache.
- D. The response quality degrades linearly but the model still processes all tokens.

Correct Answer: B

Explanation

Behaviour varies by implementation: most APIs return an error if you exceed the context limit. Some client libraries silently truncate from the beginning of the conversation. The model does not auto-summarise or retrieve — what falls outside the window is gone. Engineers must handle this explicitly.

Q37. What is "KV cache" and why does it matter for inference efficiency?

- A. A key-value store external to the model used to retrieve facts during generation.
- B. A compressed representation of the model's weights for faster loading at startup.
- C. A log of previous API calls that the model uses to maintain conversation history.
- D. A cache of the key and value matrices from previous tokens, avoiding recomputation during autoregressive generation.**

Correct Answer: D

Explanation

During autoregressive generation, each new token attends to all previous tokens. Without caching, those attention keys and values would be recomputed every step. KV cache stores them, so only the new token's computations are needed. This is essential for making generation fast — without it, long outputs would be extremely slow.

Q38. What is "context stuffing" and when is it a problem?

- A. Injecting adversarial content into the context to manipulate the model's output.
- B. Using the context window exclusively for system prompt instructions, leaving no room for user input.
- C. Filling the context with as much information as possible and hoping the model uses it — which degrades performance when the signal is buried in noise.**
- D. Compressing multiple conversations into a single context to save API costs.

Correct Answer: C

Explanation

Context stuffing — dumping large amounts of loosely relevant text into the context — seems intuitive but often hurts performance. The model struggles to identify what matters. Selective retrieval (RAG) typically outperforms context stuffing because it surfaces only the most relevant content.

Q39. How does "retrieval-augmented generation" (RAG) address context window limitations?

- A. RAG extends the model's context window by compressing retrieved documents before insertion.
- B. Instead of loading all documents into context, RAG retrieves only the most relevant chunks at query time, keeping context focused and within limits.**
- C. RAG eliminates the need for a context window by storing all knowledge in the vector database.
- D. RAG allows the model to read documents in real time from external URLs during generation.

Correct Answer: B

Explanation

RAG solves the context problem by not trying to fit everything in. A retrieval step finds the few most relevant document chunks based on the query, and only those go into the context. The model's context stays manageable and focused — improving both accuracy and cost.

Q40. What is "prompt caching" offered by some model providers?

- A. A feature that caches the computed key-value states for a shared prefix, reducing cost and latency for repeated inputs with the same preamble.**
- B. A service that stores previously generated responses and returns them verbatim for identical prompts.
- C. A client-side feature that saves prompt templates in a local file for faster reuse.
- D. A fine-tuning technique that learns which prompt structures work best for a given task.

Correct Answer: A

Explanation

Prompt caching (offered by Anthropic, OpenAI, and others) works at the infrastructure level: if many requests share the same long prefix (system prompt, document, knowledge base), the KV states for that prefix are cached and reused. This cuts both latency and cost for high-volume applications.

Q41. What does "multi-turn context management" require from an application developer?

- A. Subscribing to a stateful session tier from the model provider that maintains context server-side.
- B. Fine-tuning the model on the specific conversation history before each new session.
- C. Explicitly maintaining and passing the full conversation history with each API call, since the model has no built-in session memory.**
- D. Summarising every turn before sending it to the model to keep token counts low.

Correct Answer: C

Explanation

The model is stateless — each API call is independent. To simulate a conversation, the developer must pass the full history (user and assistant turns) with every request. This is entirely the application's responsibility. Ignoring this means each turn is answered without knowledge of what came before.

Q42. What is "context poisoning" in the context of language model security?

- A. A phenomenon where outdated training data in the model's weights causes incorrect answers.
- B. Overloading the context window with tokens to cause the model to crash or return errors.
- C. Using context length to bypass rate limits by batching many requests into one call.
- D. An attack where malicious content injected into the context manipulates the model's reasoning or outputs.**

Correct Answer: D

Explanation

Context poisoning (a form of prompt injection) occurs when adversarial content — in retrieved documents, user inputs, or external data — manipulates the model into ignoring instructions or producing harmful outputs. It is a real attack vector for RAG systems that process untrusted content.

Q43. Why does very long context not always produce better results?

- A. APIs hard-cap response quality above a certain context length to manage compute costs.
- B. Attention over very long sequences can dilute focus, and the model may fail to use distant relevant content effectively.**
- C. Long contexts always improve results — the limitation is only in the user's interpretation.
- D. Long contexts cause the model to switch to a less capable inference mode to manage memory.

Correct Answer: B

Explanation

More context is not always better. Very long contexts introduce noise, and attention mechanisms struggle to reliably use content spread across hundreds of thousands of tokens. The "lost in the middle" effect, diluted attention, and increased hallucination risk all mean that selective, focused context often outperforms raw length.

Q44. What is the practical implication of paying for both input and output tokens in most pricing models?

- A. Developers should always minimise system prompt length at the expense of instruction quality.
- B. Output tokens are always cheaper, so applications should request longer responses to get more value.
- C. Prompt design and output length both directly affect cost — verbose prompts and long responses compound expenses.**
- D. Token costs only apply to production deployments; development and testing are always free.

Correct Answer: C

Explanation

Most APIs charge per token for both input and output. A 10,000-token system prompt multiplied by millions of calls is expensive — as is requesting long responses. Efficient prompt engineering and output length control are not just quality concerns; they have direct financial implications at scale.

Q45. What is "context window utilisation" and why does it matter operationally?

- A. How much of the available context window a request actually uses — important for cost management, latency, and avoiding truncation.**
- B. A metric for how accurately the model uses information within the context to answer questions.
- C. The percentage of the training corpus that fits within the model's context window.
- D. A measure of how frequently a deployed model's context fills to capacity in production.

Correct Answer: A

Explanation

Context window utilisation tracks how close requests come to the context limit. At low utilisation, you may be under-using available context. Near the limit, you risk truncation. Monitoring this in production helps engineers right-size their prompts, manage costs, and catch unexpected growth in conversation length.

SET

4

Inference and Reasoning

Set 4 of 5 · 15 Questions · LLM Fundamentals

Q46. What does "autoregressive generation" mean?

- A. The model generates all output tokens simultaneously in a single forward pass.
- B. The model retrieves pre-generated responses from a cache and adapts them to the current input.
- C. The model repeatedly revises its entire output until a quality threshold is met.
- D. The model generates one token at a time, conditioning each new token on all previously generated tokens.**

Correct Answer: D**Explanation**

Autoregressive means each token is generated sequentially: the model takes the input plus all tokens generated so far and produces the next token. This repeats until an end token is reached or the max length is hit. It is sequential and cannot be trivially parallelised during generation.

Q47. What is "greedy decoding" and what is its main drawback?

- A. Generating multiple full responses and selecting the one with the highest overall probability.
- B. Always picking the single highest-probability token at each step — fast but can miss higher-quality sequences reachable through lower-probability early choices.**
- C. Randomly sampling tokens without any probability weighting, producing maximally diverse outputs.
- D. Constraining generation to tokens that appear in a predefined vocabulary whitelist.

Correct Answer: B**Explanation**

Greedy decoding is the simplest strategy: always pick the most likely next token. It is fast and deterministic but locally optimal — it can get stuck in repetitive or suboptimal sequences that would have been avoided by accepting a lower-probability token earlier in the sequence.

Q48. What is "chain-of-thought" reasoning and why does it improve performance?

- A. A training technique that chains multiple fine-tuning stages to build reasoning capabilities.
- B. A retrieval method that chains document lookups to gather evidence before answering.
- C. Prompting the model to reason through intermediate steps before giving a final answer, which improves accuracy on complex tasks.**
- D. A decoding strategy that generates multiple candidate chains and selects the best one.

Correct Answer: C

Explanation

Chain-of-thought (CoT) prompting asks the model to "think out loud" — showing its reasoning before the answer. This improves performance on multi-step problems (maths, logic, planning) because the model can build on intermediate steps rather than attempting to jump directly to the answer.

Q49. What distinguishes a "reasoning model" (like o1 or o3) from a standard instruction-tuned model?

- A. Reasoning models are trained to spend more compute generating internal reasoning chains before producing an answer, improving performance on hard tasks.**
- B. Reasoning models have access to external calculators and search tools by default.
- C. Reasoning models are always larger in parameter count than standard instruction-tuned models.
- D. Reasoning models generate answers in a structured JSON format rather than natural language.

Correct Answer: A

Explanation

Reasoning models (OpenAI o-series, Anthropic's extended thinking) are trained to produce extended internal reasoning — sometimes called a "scratchpad" — before the final response. This deliberate search through reasoning paths improves performance on hard tasks like maths, coding, and logical reasoning at the cost of higher latency.

Q50. What is "top-p" (nucleus) sampling?

- A. Selecting the top p percent of tokens by frequency in the training data.
- B. Limiting generation to tokens that appear in the top p documents retrieved from a knowledge base.
- C. Setting a minimum probability threshold below which tokens are never generated.
- D. Sampling from the smallest set of tokens whose cumulative probability exceeds p, avoiding both determinism and the long tail of unlikely tokens.**

Correct Answer: D

Explanation

Top-p sampling restricts the sampling pool to the nucleus — the smallest set of tokens that together account for probability mass p (e.g., 0.9 = 90%). This avoids very unlikely tokens while preserving diversity. It works well alongside temperature to produce coherent but varied outputs.

Q51. What is the difference between "latency" and "throughput" in LLM inference?

- A. Latency measures output quality; throughput measures how many servers are running simultaneously.
- B. Latency is a user-facing metric; throughput is only relevant for model training pipelines.
- C. Latency is the time from request to first token; throughput is the number of tokens or requests processed per unit time.**
- D. Latency and throughput are the same metric measured at different time scales.

Correct Answer: C

Explanation

Latency (specifically "time to first token") measures how long a user waits before seeing any output — critical for perceived responsiveness. Throughput measures total tokens generated per second across all requests — critical for cost and capacity planning. Optimising one can trade off against the other.

Q52. What is "speculative decoding" and what problem does it solve?

- A. A technique where a smaller draft model generates candidate tokens that a larger model verifies in parallel, reducing total latency.**
- B. A method where the model predicts the user's likely next prompt to pre-generate a response.
- C. A sampling strategy that generates multiple response candidates and selects the most likely one.
- D. A compression technique that reduces the model's memory footprint during inference.

Correct Answer: A

Explanation

Autoregressive generation is inherently sequential — each token must wait for the previous one. Speculative decoding breaks this by having a fast small model generate several candidate tokens ahead, then having the large model verify them in one parallel forward pass. Valid tokens are accepted; invalid ones trigger regeneration from that point.

Q53. What does "hallucination" mean in the context of LLMs?

- A. The model producing outputs that are stylistically inconsistent with the prompt.
- B. The model generating content that is confidently stated but factually incorrect or entirely fabricated.**
- C. The model repeating the same phrase or sentence multiple times in a response.
- D. The model refusing to answer questions it was trained to answer due to safety filters.

Correct Answer: B

Explanation

Hallucination is the most significant reliability failure of LLMs: the model produces plausible-sounding but false information — wrong facts, non-existent citations, fabricated statistics. It occurs because models generate probable token sequences, not because they verify truth. It is not "lying" — the model has no intent — but the effect can be just as harmful.

Q54. What is "beam search" in the context of LLM decoding?

- A. Searching the training data for the closest matching sentence to the current context.
- B. Filtering generated tokens through a classifier beam before adding them to the output.
- C. Maintaining multiple candidate sequences simultaneously and selecting the one with the highest overall probability at the end.**
- D. Splitting generation across multiple GPU cores to increase throughput.

Correct Answer: C

Explanation

Beam search keeps k candidate sequences (beams) at each step, expanding each with the top tokens and pruning back to k . It explores more of the output space than greedy decoding, often producing higher-probability sequences. However, high-quality diverse generation has largely shifted to sampling with temperature and top- p .

Q55. Why do reasoning models have higher latency than standard models?

- A. They make multiple API calls to external tools before generating a response.
- B. They run each response through a separate verification model before returning it.
- C. They are deployed on fewer servers than standard models, creating longer queue times.
- D. They generate extended internal reasoning chains — often thousands of tokens — before producing the visible answer.**

Correct Answer: D

Explanation

The latency cost of reasoning models comes from the internal "thinking" tokens — the model generates a full reasoning trace, sometimes thousands of tokens long, which is not always shown to the user. This computation takes time. The trade-off: much better accuracy on hard tasks, but users wait longer.

Q56. What does "token budget" mean in practical LLM application design?

- A. The allocation of available context tokens across system prompt, retrieved content, conversation history, and output, managed to stay within limits and control cost.**
- B. The total number of tokens a user is permitted to generate per month under a subscription plan.
- C. A training-time constraint that limits how many tokens the model can generate during fine-tuning.
- D. A safety mechanism that stops generation when the model's confidence falls below a threshold.

Correct Answer: A

Explanation

Token budget thinking is an engineering discipline: you have a fixed context window and a cost per token. You must allocate those tokens across competing needs — instructions, examples, retrieved docs, history, response space. Good application designers track this explicitly, not as an afterthought.

Q57. What is "few-shot inference" and how does it differ from fine-tuning?

- A. Few-shot inference requires fewer API calls than fine-tuning and is therefore cheaper.
- B. Few-shot inference provides examples in the prompt to guide the model at runtime; fine-tuning updates the model's weights through additional training.**
- C. Few-shot inference works only with small models; fine-tuning is required for large models.
- D. Few-shot inference and fine-tuning are synonymous — both adapt the model to new tasks.

Correct Answer: B

Explanation

Few-shot inference adapts model behaviour by including examples in the prompt — no weight updates occur. Fine-tuning actually updates the model's parameters through gradient descent on new training data. Few-shot is faster and cheaper to try; fine-tuning is more durable but requires data, compute, and care.

Q58. What is "output consistency" and why is it a challenge with temperature > 0?

- A. Output consistency is guaranteed when using the same model version and system prompt.
- B. Output consistency only matters for creative tasks; factual tasks are always deterministic.
- C. Output consistency can be enforced by setting max_tokens to a fixed value.
- D. At temperatures above 0, the model samples probabilistically, so identical inputs can produce different outputs — inconsistency is inherent.**

Correct Answer: D

Explanation

At temperature 0, decoding is nearly deterministic (greedy). At temperature > 0, each run samples from the probability distribution, so outputs vary. For applications requiring reproducibility — structured data extraction, classification — low temperature or temperature 0 is preferred, accepting reduced diversity.

Q59. What is the "prefill" phase of LLM inference?

- A. The phase where the model processes the full input prompt in parallel, before beginning autoregressive generation.**
- B. The phase where the server pre-loads model weights into GPU memory before the first request.
- C. The phase where the application populates template placeholders before sending the prompt.
- D. The phase where retrieved documents are ranked and inserted into the context window.

Correct Answer: A

Explanation

Inference has two phases. Prefill: the model processes the entire input prompt in a single (parallelisable) forward pass, generating KV cache for all input tokens. Decode: the model generates output tokens one at a time autoregressively. Prefill is fast; the decode phase is where latency compounds.

Q60. What does "model quantisation" do and why is it used?

- A. It compresses the model by removing parameters that contributed least to training performance.
- B. It converts the model from one architecture to another for deployment on different hardware.
- C. It reduces the numerical precision of model weights (e.g., from float32 to int8), shrinking memory footprint and increasing inference speed with modest quality trade-offs.**
- D. It splits the model into smaller modules that can be fine-tuned independently.

Correct Answer: C

Explanation

Quantisation reduces the bit-width of model weights — from 32-bit or 16-bit floats to 8-bit or 4-bit integers. The model becomes smaller and faster to run, at the cost of some precision. It is widely used to deploy large models on consumer GPUs or to reduce cloud inference costs.

SET

5

Model Selection and Fine-tuning

Set 5 of 5 · 15 Questions · LLM Fundamentals

Q61. What is the primary consideration when choosing between a large frontier model and a smaller efficient model for a production application?

- A. Task complexity, latency requirements, cost constraints, and privacy needs — not parameter count alone.
- B. The larger model is always preferable because it will produce better outputs.
- C. Model selection should always prioritise the most recently released model.
- D. The choice depends entirely on the context window size required for the application.

Correct Answer: A

Explanation

Model selection is a multi-factor engineering decision. A large model may outperform on complex reasoning but cost 10–100x more per token and add latency. Smaller models may be sufficient for extraction, classification, or structured output tasks. Privacy, latency, and cost shape the decision as much as capability.

Q62. What is "fine-tuning" and when is it the right choice over prompting?

- A. Fine-tuning is always the first step in deploying an LLM for any enterprise application.
- B. Fine-tuning is only possible for open-source models; commercial models cannot be fine-tuned.
- C. Fine-tuning is a synonym for instruction tuning — it refers to the initial alignment step performed by the model provider.
- D. Fine-tuning updates model weights on new training data — appropriate when prompt engineering cannot achieve the required consistency, style, or task performance.

Correct Answer: D

Explanation

Fine-tuning is appropriate when you have high-quality task-specific data and need durable behaviour that prompt engineering cannot reliably achieve. Examples: enforcing a specific output format, learning domain vocabulary, or replicating a tone at scale. Start with prompting; fine-tune only when you have evidence it is needed.

Q63. What is a "base model" versus an "instruction-tuned model"?

- A. A base model is smaller; an instruction-tuned model is the same architecture with more parameters.
- B. A base model is trained only on next-token prediction; an instruction-tuned model has been further trained to follow instructions and behave helpfully.**
- C. A base model is free to use; an instruction-tuned model requires a paid API subscription.
- D. A base model generates code; an instruction-tuned model generates natural language.

Correct Answer: B

Explanation

Base models (like GPT-4 base or Llama before instruction tuning) are powerful but unpredictable in conversation — they complete text, not follow instructions. Instruction-tuned models (Claude, ChatGPT, etc.) have been aligned via supervised fine-tuning and RLHF to be helpful, follow user intent, and behave appropriately.

Q64. What is "LoRA" (Low-Rank Adaptation) and why is it used for fine-tuning?

- A. A regularisation technique that penalises large weight updates during fine-tuning to prevent catastrophic forgetting.
- B. A data augmentation method that generates synthetic training examples using the base model.
- C. A parameter-efficient fine-tuning method that injects small trainable rank-decomposition matrices into the model, leaving original weights frozen — dramatically reducing compute and memory requirements.**
- D. A deployment optimisation that reduces the number of active layers during inference.

Correct Answer: C

Explanation

Full fine-tuning of a large model requires updating billions of parameters — expensive and prone to catastrophic forgetting. LoRA freezes the original weights and adds small trainable matrices at specific layers. These adapters can be trained cheaply, swapped between tasks, and merged into the base model at deployment.

Q65. What is "catastrophic forgetting" in the context of fine-tuning?

- A. When fine-tuning on new data causes the model to lose performance on tasks it previously handled well.**
- B. When the model generates outputs that directly contradict its training data.
- C. When a model's fine-tuning run fails midway and all checkpoints are lost.
- D. When a model overfits to the fine-tuning data and performs worse than the base model on the target task.

Correct Answer: A

Explanation

Catastrophic forgetting occurs when gradient updates for new tasks overwrite the weight configurations that enabled prior capabilities. A model fine-tuned aggressively on legal documents might lose its general reasoning ability. Techniques like LoRA, rehearsal, and regularisation are used to mitigate this.

Q66. What is a "system prompt" in the context of deployed LLM applications?

- A. A special prompt that triggers the model to enter a system administration mode.
- B. A hidden prompt generated by the model itself to guide its own reasoning.
- C. A fixed instruction block provided by the operator that sets the model's behaviour, persona, and constraints before the user's input is processed.**
- D. A prompt constructed automatically from the user's account metadata and preferences.

Correct Answer: C

Explanation

The system prompt is the operator's layer of control — instructions the model receives before any user message. It defines persona, sets rules, restricts topics, or provides context. It persists across the conversation and is typically not visible to the end user. It is the primary tool for customising model behaviour without fine-tuning.

Q67. What factors should drive the decision to use an open-source model versus a commercial API?

- A. Open-source models are always cheaper; commercial models are always more capable.
- B. Commercial APIs are suitable for prototypes; open-source models should always be used in production.
- C. The choice depends only on the licence type of the open-source model.
- D. Data privacy requirements, customisation needs, cost at scale, latency constraints, and available ML engineering capacity.**

Correct Answer: D

Explanation

Open-source models (Llama, Mistral, Qwen) offer full control, no data leaving your infrastructure, and no per-token costs at scale — but require significant ML engineering to deploy and maintain. Commercial APIs are faster to start with and continuously updated but involve data sharing, vendor lock-in, and ongoing costs.

Q68. What is "domain adaptation" in the context of LLMs?

- A. Translating a model trained on English data to perform well on other language domains.
- B. Adjusting a general-purpose model — through fine-tuning, prompting, or RAG — to perform well on a specific domain such as legal, medical, or financial text.**
- C. Adapting a text model to handle non-text modalities like images or audio.
- D. Changing the model's system prompt to make it sound like a domain expert.

Correct Answer: B

Explanation

Domain adaptation addresses the gap between a general-purpose model's training data and a specific domain's vocabulary, reasoning patterns, and knowledge. This can be achieved through domain-specific fine-tuning, retrieval of domain documents, or structured prompting — each with different cost-quality trade-offs.

Q69. What is "multimodal" capability in an LLM?

- A. The ability to output multiple response formats such as JSON, Markdown, and plain text.
- B. The ability to handle multiple languages in the same model.
- C. The ability to process and generate content across multiple modalities — text, images, audio, or video — within a single model.**
- D. The ability to run on multiple hardware platforms including CPU, GPU, and TPU.

Correct Answer: C

Explanation

Multimodal models extend the token paradigm beyond text. Vision-language models (GPT-4o, Claude 3, Gemini) can process images alongside text. Some models handle audio or video. This enables use cases like document analysis, image description, diagram interpretation, and video understanding.

Q70. What is the significance of a model's "knowledge cutoff"?

- A. The model has no reliable knowledge of events that occurred after its training data cutoff date.**
- B. The model refuses to answer questions that require more than its knowledge cutoff token limit.
- C. The knowledge cutoff determines the maximum context the model can process.
- D. The knowledge cutoff is the date after which the model's licence expires.

Correct Answer: A

Explanation

A knowledge cutoff is the date beyond which training data was not collected. Ask a model about events after its cutoff and it either doesn't know or hallucinates. This is a critical limitation for real-time applications — addressed by RAG, tool use for web search, or regularly re-training/updating the model.

Q71. What is the difference between "zero-shot" and "few-shot" prompting in the context of model capability?

- A. Zero-shot prompting works only on fine-tuned models; few-shot prompting works on base models.
- B. Zero-shot asks the model to perform a task without examples; few-shot provides examples in the prompt to demonstrate the desired behaviour.**
- C. Zero-shot produces shorter outputs; few-shot produces longer outputs.
- D. Zero-shot is faster because it skips the example processing step.

Correct Answer: B

Explanation

Zero-shot: the model is asked to do something with no examples — relying entirely on its pre-trained knowledge and instruction-following ability. Few-shot: the prompt includes worked examples showing input-output pairs, demonstrating the desired format or reasoning pattern. Few-shot generally improves performance on structured tasks.

Q72. What is "model distillation" and what is it used for?

- A. Extracting factual knowledge from a large model into a structured database.
- B. Removing unsafe capabilities from a model by filtering its outputs during training.
- C. Splitting a large model into smaller independent sub-models for parallel deployment.
- D. Training a smaller student model to mimic the outputs of a larger teacher model — producing a compact model that retains much of the teacher's capability.**

Correct Answer: D

Explanation

Distillation trains a small model on the output distribution of a large model — the small model learns not from raw data labels, but from the teacher's soft probability distributions and reasoning patterns. The result is a smaller, faster, cheaper model that retains much of the large model's quality on target tasks.

Q73. When is fine-tuning NOT the right solution?

- A. When the task involves structured output — fine-tuning never helps with format compliance.
- B. When using a commercial model, since fine-tuning is only available for open-source models.
- C. When you have limited high-quality data, when the issue is solvable with better prompting, or when you need the model to use knowledge that post-dates training.**
- D. When the application requires low latency, since fine-tuned models are always slower.

Correct Answer: C

Explanation

Fine-tuning is often reached for prematurely. If the underlying issue is a bad system prompt, lack of examples, or missing context — these are prompting and RAG problems, not fine-tuning problems. And fine-tuning cannot inject new post-training knowledge — that requires retrieval. Validate that prompting cannot solve the problem first.

Q74. What does "safety alignment" mean in the context of commercial LLMs?

- A. Encrypting model weights to prevent extraction or misuse by third parties.
- B. Training processes — including RLHF, Constitutional AI, or similar — that teach the model to refuse harmful requests and behave in accordance with defined values.**
- C. Restricting the model's outputs to a whitelist of approved response types.
- D. Testing the model against a fixed benchmark to ensure outputs are factually correct.

Correct Answer: B

Explanation

Safety alignment refers to the deliberate training steps that shape a model's values and behaviour — teaching it to be helpful, honest, and avoid causing harm. Anthropic uses Constitutional AI and RLHF; OpenAI uses similar methods. Alignment is not a filter on top of the model — it is baked into the weights.

Q75. What is the practical implication of model deprecation for production applications?

- A. Deprecated models become freely available for local deployment once removed from the API.
- B. Deprecation only affects fine-tuned models; base model API access is permanent.
- C. Model deprecation triggers automatic migration to the latest version with no engineering work required.
- D. Applications built on a deprecated model version must be re-evaluated and potentially re-engineered when that version is removed from the API.**

Correct Answer: D

Explanation

Model providers regularly deprecate older versions — sometimes with weeks of notice. Applications that depend on specific model behaviour (format, capability, safety) must be monitored, re-tested against successor models, and potentially re-engineered. Deprecation risk is a real operational concern for production AI systems.



Your turning point in the AI era.

END OF QUESTION BANK

*Your turning
point starts
here.*